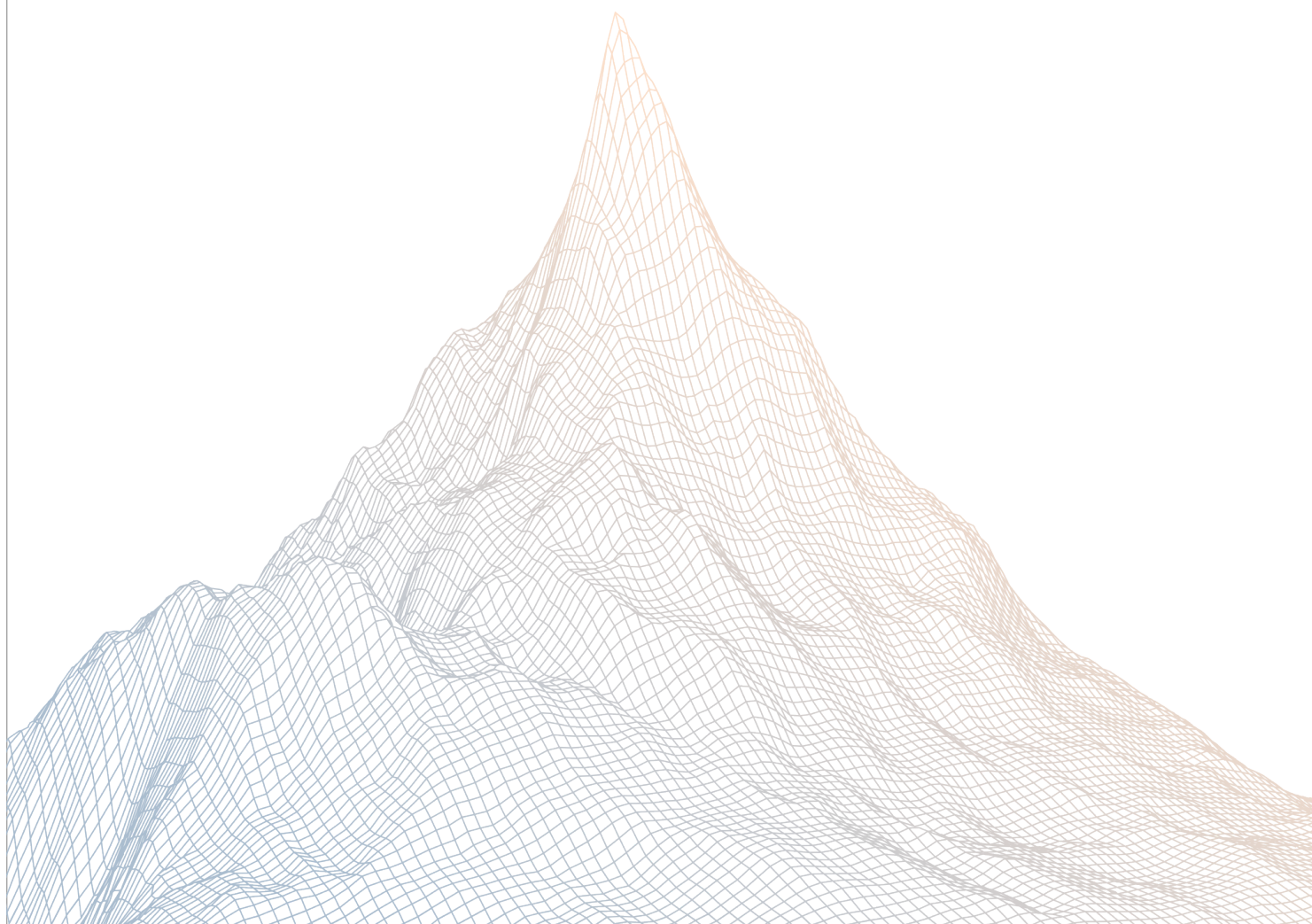


Lattica

Smart Contract Security Assessment

VERSION 1.1



Contents

1	Introduction	3
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	4
2.1	About Lattica	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	7
4	Findings	9
4.1	High Risk	9
4.2	Medium Risk	18
4.3	Low Risk	38

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Lattica

Serving as the liquidity layer of informational finance, Lattica offers margin and credit against prediction market exposure: position holders draw liquidity against ERC1155 outcome tokens they already hold, and traders open amplified exposure to a market outcome against posted margin.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts
---------------	-----------

Repository	https://github.com/latticafi/contracts
-------------------	---

Commit Hash	5fd0dd73a558362f5d57b5900c561e4ef739d15b
--------------------	--

Files	src/**/*.sol
--------------	--------------

Target	contracts Mitigation Review
---------------	-----------------------------

Repository	https://github.com/latticafi/contracts
-------------------	---

Commit Hash	f4848bfc13a181cd0dcb3df7d126dd1d2692e175
--------------------	--

Files	src/**/*.sol
--------------	--------------

Target	contracts
Repository	https://github.com/latticafi/contracts/pull/17
Commit Hash	64b3d295cff76e78e9df21ee93615270cf1758b2
Files	Changes in PR-17

2.3 Audit Timeline

September 16, 2026	Audit start
September 26, 2026	Audit end
October 2, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	4
Medium Risk	10
Low Risk	5
Informational	0
Total Issues	19

3

Findings Summary

ID	Description	Status
H-1	Traders can withdraw the margin that keeps a leverage position safe	Resolved
H-2	New deposits can capture recoveries from losses borne by existing lenders	Resolved
H-3	Early stream settlement lets new depositors capture pre-paid lender income	Resolved
H-4	A zero-valued share remainder stalls subsequent redemptions	Resolved
M-1	Borrowers can roll loans while lenders are waiting for cash	Resolved
M-2	Dust repayments by anyone can break a borrower's signed roll	Resolved
M-3	Loan payouts can spend cash reserved for pending withdrawals	Resolved
M-4	Recovery can pay the treasury while lenders still have an unrecovered loss	Resolved
M-5	Renewal can keep debt above the current collateral limit	Resolved
M-6	Renewal is rejected when the existing loan already fills the budget	Resolved
M-7	Temporary liquidity can lock in cheaper interest for a loan epoch	Resolved
M-8	Leverage reservations count the principal twice when checking available liquidity	Resolved
M-9	Repayment after cluster reassignment removes unrelated exposure from budget accounting	Resolved
M-10	Capped recoveries make the loss ratio depend on settlement order	Resolved

ID	Description	Status
L-1	Recovery after all shares exit can trap the next depositor below the redemption floor	Resolved
L-2	A zero-premium loss can leave the automatic breaker open	Resolved
L-3	A delayed expiry claim lets the first LP redeem at an inflated price	Resolved
L-4	Leverage advances cannot be reconciled through the reserve repayment path	Resolved
L-5	A negligible early default can latch the pool's admission breaker	Resolved

4

Findings

4.1 High Risk

A total of 4 high risk findings were identified.

[H-1] Traders can withdraw the margin that keeps a leverage position safe

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [Leverage.sol#L279-L321](#)
- [Leverage.sol#L168-L204](#)
- [margin-ops.md#L37-L46](#)
- [margin-ops.md#L57-L75](#)
- [margin-ops.md#L90-L103](#)

Description:

A leverage position's margin is the first cash used to absorb a shortfall when the executor closes the position. Extra margin also gives the executor more room before it has to close the position, because the documented health rule counts both the sold shares and the margin.

The issue appears when the market has already moved down, but the extra margin still keeps the position above the close threshold. At that point, `removeMargin()` can let the trader remove the same margin cushion that kept the position safe. The onchain check is only a static floor based on the position debt and the market LTV band; it does not use the current mark. Once the static check passes, the contract sends the removed pUSD to the trader wallet:

```
uint256 required = _marginRequirement(p);
uint256 margin = p.margin;
uint256 remaining = amount > margin ? 0 : margin - amount;
if (amount > margin || remaining < required) revert
    Errors.MarginBelowRequirement(posId, required, remaining);

uint256 newMargin = ledger.removePositionMargin(posId, amount);
emit Events.MarginRemoved(posId, p.trader, amount, newMargin);
```

```
pusd.safeTransfer(p.trader, amount);
```

This floor calculation never reads the current price:

```
uint256 band = configurator.bandLtv(p.tokenId);
if (band == 0) return type(uint256).max;
return MathLib.mulDivUp(p.notional, MathLib.BPS - band, band);
```

When the position later closes, the contract can only use the margin that is still recorded on the position. Margin already withdrawn to the trader wallet is no longer available to absorb the shortfall:

```
uint256 marginAbsorbed = shortfall < p.margin ? shortfall : p.margin;
uint256 stillShort = shortfall - marginAbsorbed;

ledger.closePosition(posId, returned, marginAbsorbed + bufferCash, fronted);

if (marginAbsorbed != 0) pusd.safeTransfer(address(vault), marginAbsorbed);
```

For example:

1. A trader has 8,000 shares, 4,000 pUSD debt, and 1,000 pUSD margin. At the 80% LTV band, the static floor is 1,000 pUSD.
2. The trader adds another 1,000 pUSD of margin. This gives the executor more price buffer before the position should be closed.
3. The mark falls to 0.350 pUSD per share. With 2,000 pUSD margin, the position has $8,000 * 0.350 + 2,000 = 4,800$ pUSD against the required 4,400 pUSD safety value, so it can remain open.
4. The trader removes 1,000 pUSD. The static floor still passes, but the same mark now leaves $8,000 * 0.350 + 1,000 = 3,800$ pUSD, below both the required 4,400 pUSD safety value and the 4,000 pUSD debt.
5. If the executor later closes at the same mark, only the remaining 1,000 pUSD margin is available. The extra margin that would have covered the loss has already been returned to the trader wallet, so the remaining shortfall reaches the protocol waterfall.

This requires a trader-wallet path that lets the user request `removeMargin()` and use the returned pUSD by withdrawing, transferring, or reusing it. If each withdrawal needs executor or health-service approval that checks the current mark and rejects unsafe withdrawals before signing, that approval is the control that prevents this user path.

Recommendations:

Do not rely on the static margin floor alone. Require executor or health-service approval before margin can be removed. The approval should confirm that the position stays safe after the withdrawal; otherwise the position should be closed or locked before any margin leaves.

Lattica: Resolved with [@b050a87758...](#)

Zenith: Verified.

[H-2] New deposits can capture recoveries from losses borne by existing lenders

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/LatUSD.sol](#)
- [src/Ledger.sol](#)
- [src/Liquidator.sol](#)

Description:

A lender can acquire shares at a price that excludes a forthcoming recovery and then participate in that recovery without having held shares when the associated loss was recognized. When Ledger settles a liquidated or expired loan, it removes the principal receivable, credits the covered amount to idle assets, and records the uncovered balance in the loss statistic. Any later disposal proceeds are absent from the NAV used for new deposits. The accrual performed by LatUSD before pricing a deposit updates streamed income; it does not account for those future proceeds.

An honest, authorized remitter subsequently supplies actual pUSD through Liquidator.remitProceeds. The contract first repays the outstanding Buffer advance and transfers the treasury allocation. Only the remainder, poolShare, reaches the vault and increases Ledger's idle balance through recordDeposit. This credit raises the value of the shares outstanding at that time without issuing additional shares. Reducing realizedLosses through recordRecovery only updates a statistic and does not distinguish holders who experienced the writeoff from later entrants.

Consequently, an ordinary lender with sufficient funds and successful ordering ahead of a visible remittance can receive a material part of the residual recovery. The lender needs no remitter privileges. Existing shareholders receive a smaller fraction of that payment than they would without the intervening entry. The contracts intentionally allocate the credit through current share ownership and maintain no individual recovery entitlement for former holders; the economic concern is that temporary ownership diverts value associated with the previously impaired book, contrary to the documented objective that deposit and withdrawal timing should not extract value.

The amount available for this effect is limited to the residual paid into the pool. Buffer repayment and treasury funds are excluded; no surplus, or a treasury allocation of the entire surplus, leaves nothing to capture. For deposit D, NAV A immediately before entry, and

residual recovery R , the share conversion formulas imply a gross gain bounded by $D \times R / (A + D + 1)$ on an otherwise unchanged path with an empty queue. Ignoring the virtual asset and rounding, this approaches $D \times R / (A + D)$, so capturing most of R requires capital substantially larger than A .

Realizing the uplift still depends on a redemption request meeting the minimum size, FIFO fulfillment, and enough idle liquidity. Until fulfillment, the shares remain exposed to intervening losses and delays; redemption only transfers assets already assigned to the controller. Allocation becomes final when paid. This does not establish a costless or flash-financed withdrawal, and financing costs, gas, and interim exposure can erase the benefit. The opportunity depends on an actual funded recovery with positive pool proceeds; its frequency, size, and net profitability are not established.

Recommendations:

Ensure that all share entry, recovery-credit, and redemption boundaries preserve the economic allocation of recoveries associated with losses recognized before entry, so acquiring temporary shares cannot divert that value from the loss-bearing book.

Lattica: Resolved with [@c33caleee7....](#)

Zenith: Verified.

[H-3] Early stream settlement lets new depositors capture pre-paid lender income

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/BorrowerLogic.sol](#)
- [src/LatUSD.sol](#)
- [src/Ledger.sol](#)
- [src/Leverage.sol](#)

Description:

Share issuance and early stream settlement apply different valuations to the same prepaid income. `Ledger._totalAssets` includes idle cash, principal and executor receivables, and income already accrued through the index. It excludes the portion of an active stream's reservoir that has not accrued. Although `LatUSD.deposit` synchronizes accrual before calculating shares, that synchronization recognizes only elapsed streaming time; it does not include the remaining prepaid amount in the entry price.

A full repayment takes a different approach. `recordRepay` invokes `_stopStream` with `lumpRemainder` enabled once the outstanding principal reaches zero. The stream's accrued contribution is removed from the index, but its entire prepaid lender share is added to idle. Consequently, the previously unrecognized remainder increases NAV immediately. Exchanging repaid principal for idle is value-neutral; recognizing this extra income is not. Shares issued shortly before settlement participate in that increase, diverting part of the existing holders' income allocation to the entrant.

The public repayment entry point accepts payment from any address. A funded depositor can therefore cause this recognition when an active loan already has a sufficiently small unpaid balance. The payer must actually supply that balance, and the collateral transfer back to the recorded borrower must succeed. No collateral ownership or principal debt is transferred to the depositor. The same stream-stop behavior occurs on a loan roll and a leverage close, allowing entry timing around another party's observable, honest settlement when it produces a positive NAV change. Rolls require the borrower, and leverage closes require the executor; a depositor has neither authority over another borrower's roll nor the executor's close privilege. A close that realizes losses can consume or outweigh the income increase.

The resulting shares can receive the higher valuation through the normal redemption queue,

whose fulfillment prices at current NAV. Realizing a gain still requires sufficient idle liquidity, a favorable FIFO position, and a request above the configured minimum. Separate completed calls suffice; reentrancy is unnecessary. Waiting for liquidity can delay withdrawal and leave the shares exposed to intervening losses.

Economically, if the entrant owns fraction f when a positive unrecognized amount E becomes recognized, the attributable gross benefit is bounded by $f \times E$, subject to virtual-share accounting and rounding. Net benefit must also deduct every unit of principal the entrant paid, financing expense, and transaction costs. Capturing nearly all of E requires capital sufficient to dominate share ownership. Reducing a large debt personally and then paying its final small balance does not make the earlier payments free. For a newly originated direct loan, the remaining income cannot exceed the lender's charge allocation, which cannot exceed total charges or the original principal; paying that whole original debt cannot profit solely from this income effect.

The affected value is the vault's prepaid income at qualifying early settlements. Practical profitability depends on substantial remaining income and either an independently inexpensive residual debt or usable visibility into an honest settlement; their availability and financing terms are not established here. Recognizing the full reservoir is intentional under the charge-retention rule, but combining it with an entry price that omits the remainder defeats the documented protection against extracting value through deposit and withdrawal timing.

Recommendations:

Ensure that share pricing and income recognition preserve the existing lenders' allocation across every repayment, roll, and close that ends a stream early. Prevent newly entering capital from receiving a windfall solely because prepaid income is recognized at settlement rather than included in its entry valuation.

Lattica: Resolved with [@4b1a9557d1...](#)

Zenith: Verified.

[H-4] A zero-valued share remainder stalls subsequent redemptions

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/LatUSD.sol](#)
- [src/Ledger.sol](#)
- [src/libraries/MathLib.sol](#)

Description:

LatUSD processes redemption requests in FIFO order, but its partial-fulfillment path can leave the first request with shares too small to exchange for one asset base unit. That request then prevents the vault from fulfilling every request behind it, including after spendable cash becomes available. The entry check against `minRequest` applies to the original request; it does not constrain the balance left after a partial fill.

Inside `_crank`, the vault limits the shares to fulfill by the amount that idle cash can cover, then converts those shares to assets with downward rounding. When a partial fill leaves a positive remainder, the request stays at `_head`. On a later call, even enough idle cash to cover that remainder cannot produce a nonzero `fillAssets` if its rounded asset value is zero. The loop exits at that check, before updating fulfillment, reducing `queuedShares`, or advancing the cursor.

For a remaining share quantity d , tracked assets A , total share supply S , and virtual shares V , the conversion is $\text{floor}(d \times (A + 1) / (S + V))$. The obstruction therefore persists while $d \times (A + 1) < S + V$. Repaying principal can merely move value from receivables into idle, and deposits accompanied by proportional share issuance need not materially change this ratio. Deposits can consequently succeed without restoring queue progress. Sufficient appreciation in assets per share can eventually make the remainder payable: with a six-decimal asset and a single raw share left near par, the required ratio increase is approximately a trillionfold. The vault exposes no request cancellation or head-skip mechanism, and changing the request minimum or batch ceiling does not resolve an existing remainder.

An ordinary shareholder can encounter or deliberately create this condition using owned shares and the permissionless `crank`, provided the request meets the configured minimum and reaches the front at a suitable liquidity boundary. This depends on share price, queue position, and timing; it is not a claim that a particular live queue is already blocked. Although the leftover shares can be dust, the initial holding may need to be comparable in

value to the available idle cash. The shareholder can collect the portion already fulfilled while the remainder continues to obstruct later lenders. Previously assigned assets remain accessible through the independent claim paths; the failure denies pending exits rather than transferring other lenders' funds to the shareholder.

The stalled requests also continue contributing to `queueLiability`, which reserves idle cash against new originations. A lone positive remainder that rounds down to zero rounds up to just one asset base unit for this reservation, so a substantial lending restriction depends on further shares accumulating behind it. The direct consequence is loss of exit availability for subsequent pending lenders in the affected vault, without a direct profit from the obstruction itself.

Recommendations:

Ensure every queue fulfillment transition preserves forward progress when an unpaid share remainder is worth less than one asset unit, so later requests can still be serviced. Preserve lender entitlements and consistent pending-share, claim, and liquidity-reservation accounting across all such transitions.

Lattica: Resolved with [@e8afc97339....](#)

Zenith: Verified.

4.2 Medium Risk

A total of 10 medium risk findings were identified.

[M-1] Borrowers can roll loans while lenders are waiting for cash

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [BorrowerLogic.sol#L97-L131](#)
- [Ledger.sol#L205-L230](#)
- [LatUSD.sol#L196-L226](#)
- [LatUSD.sol#L422-L432](#)
- [vault-design.md#L136-L146](#)

Description:

A lender can request redemption even when the vault does not have enough idle cash to pay immediately. The lender's shares enter the first-in, first-out queue and wait for cash. The protocol's design says queued lenders are senior to new borrowing, and that capital already requested for withdrawal should not be used for new risk.

An active borrower can still roll a loan using a fresh quote. The roll replaces the old loan with a new loan for the same principal, without returning that principal as idle cash for the queued lender. `rollLoan()` applies the normal market and quote checks but does not check the pending redemption liability. The ledger credits the old principal and immediately originates its replacement:

```
idle += outstanding;
_stopStream(loanId, true);

newLoanId = _originate(borrower, tokenId, qty, q);
```

For example:

1. Lenders deposit 10,000 pUSD. Alice borrows 3,000 pUSD for a 24-hour term, leaving

7,000 pUSD idle in the vault.

2. Twelve hours later, a lender requests redemption of shares worth about 8,015 pUSD. The request succeeds, but it remains pending because the vault has only 7,000 pUSD idle.
3. Alice uses a fresh roll quote for another 24-hour term. The old 3,000 pUSD loan is replaced with a successor loan, so the principal is not returned when the original loan would have ended.
4. When the original loan would have ended 12 hours later, the queued lender still has no claimable cash from this loan. The successor loan now has another 12 hours before it ends.

The result is that a borrower can keep using principal that would otherwise have become available to an already queued lender.

Recommendations:

Reject loan rolls while redemption requests are waiting for cash. A roll should not be allowed when queued redemption liability is greater than idle cash, because the old principal is still needed to serve the queue.

Lattica: Resolved with [@1d7552ce1f...](#)

Zenith: Verified.

[M-2] Dust repayments by anyone can break a borrower's signed roll

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [BorrowerLogic.sol#L73-L94](#)
- [BorrowerLogic.sol#L97-L116](#)
- [Ledger.sol#L237-L255](#)
- [Liquidator.sol#L238-L245](#)

Description:

Direct loans allow anyone to repay on behalf of the borrower. This is useful for integrations, but even a tiny partial repayment changes the loan's stored principal when the payer is not the borrower:

```
ledger.recordRepay(loanId, amount);
pusd.safeTransferFrom(msg.sender, address(vault), amount);
```

The ledger immediately subtracts the amount from the outstanding principal:

```
uint256 remaining = outstanding - amount;
l.principal = remaining.toUint128();
idle += amount;
totalPrincipalOutstanding -= amount;
```

A roll quote must match the current principal exactly. After a dust repayment by a third party, a quote signed for the previous principal reverts before the borrower can use it:

```
if (q.principal != l.principal) revert
Errors.PrincipalOutOfBand(l.principal, q.principal);
```

For example:

1. Alice has a 3,000 pUSD loan that is about to expire.

2. Alice obtains a valid roll quote for exactly 3,000 pUSD.
3. A third party pays 1 raw pUSD unit, which is 0.000001 pUSD because pUSD has 6 decimals.
4. The loan principal becomes 2,999.999999 pUSD.
5. Alice's roll quote for 3,000 pUSD now fails with a principal mismatch.
6. If this happens at the end of the term and Alice cannot get a replacement quote in time, anyone can claim the loan as expired and move the collateral into the liquidation path.

The third party pays real pUSD, and the borrower can recover by getting a fresh quote if there is enough time. The risk is the last-moment window where the borrower is relying on an already signed roll for the old principal.

Recommendations:

Prevent unrelated accounts from changing the principal during the borrower's roll window. One direct option is to allow third-party repayment only for full repayment, or to require the borrower to opt in to third-party partial repayments. Another option is to let roll accept a small signed tolerance around the current principal, so a one-unit repayment cannot break an otherwise valid borrower roll.

Lattica: Resolved with [@3527713704...](#)

Zenith: Verified.

[M-3] Loan payouts can spend cash reserved for pending withdrawals

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [BorrowerLogic.sol#L64-L70](#)
- [LatUSD.sol#L345-L354](#)

Description:

Cash owed to pending withdrawal requests should be reserved before the pool makes new loans. A loan's total use of that cash must fit within the amount available for new lending.

Direct borrowing makes two payouts: the borrower's proceeds after fees and the Buffer's fee share. Both payouts independently check the same available amount. The first payout does not reduce that allowance before the second check, so their combined payment can spend cash reserved for withdrawals.

In the code below, each `disburse()` checks the available lending cash, but the loan is recorded only after both payments. Since neither payment updates Ledger idle, the second check uses the same allowance as the first:

```
vault.disburse(p.receiver, q.principal - charges);  
if (bufferCut != 0) vault.disburse(buffer, bufferCut);  
loanId = ledger.openLoan(msg.sender, p, q);
```

For example:

1. The vault has 10,000 pUSD idle, with 3,000 pUSD reserved for pending withdrawals. This leaves 7,000 pUSD available for lending.
2. An 8,000 pUSD loan deducts 1,000 pUSD of fees: 100 goes to the Buffer and 900 stays in the vault for gradual lender income. The borrower receives the remaining 7,000 pUSD.
3. The borrower payment passes the 7,000 pUSD availability check. The 100 pUSD Buffer payment also passes because the first payment has not reduced the recorded allowance.
4. The two payments total 7,100 pUSD, exceeding available lending cash by 100 pUSD and using money reserved for withdrawals.

Recommendations:

Before either payout, check the full principal against the amount available for new lending. This covers both outgoing payments and the fees retained outside idle cash, and leaves the withdrawal reservation intact:

```
uint256 available = vault.availableForOrigination();
if (q.principal > available) {
    revert Errors.ExceedsAvailableForOrigination(q.principal, available);
}
vault.disburse(p.receiver, q.principal - charges);
if (bufferCut != 0) vault.disburse(buffer, bufferCut);
```

Lattica: Resolved with [@7c6ca9df82...](#)

Zenith: Verified.

[M-4] Recovery can pay the treasury while lenders still have an unrecovered loss

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [Liquidator.sol#L150-L176](#)
- [Ledger.sol#L637-L649](#)
- [lattica-protocol-spec.md#L149-L158](#)

Description:

The documented recovery policy repays the Buffer's advance first, then splits all remaining cash between the treasury and the pool. This split does not first check whether lenders have recovered the loss they suffered at liquidation.

As a result, recovering the loan's entire principal does not necessarily make the pool whole. The treasury can receive revenue while lenders remain short. This follows the stated policy; it is an economic-policy recommendation rather than a failure to implement that policy.

The code below treats all recovery beyond the Buffer advance as surplus and immediately gives the treasury its configured share. It does not first deduct the pool's unrecovered principal loss:

```
uint256 surplus = amount - bufferRepaid;
uint256 treasuryCut = MathLib.bpsDown(surplus, surplusSplitBps);
uint256 poolShare = surplus - treasuryCut;
```

For example:

1. A 1,000 pUSD loan settles with a 200 pUSD Buffer advance. The pool bears the remaining 800 pUSD loss.
2. Selling the collateral later recovers the full 1,000 pUSD principal.
3. The Buffer receives 200 pUSD. With a 20% treasury split, the remaining 800 pUSD pays 160 pUSD to the treasury and 640 pUSD to the pool.
4. Lenders recover only 640 pUSD of their 800 pUSD loss and remain 160 pUSD short, despite full principal recovery.

Recommendations:

If lender principal should be restored before treasury revenue is paid, track the unrecovered pool loss for each settled loan. After repaying its Buffer advance, restore that loan's pool loss before splitting any remaining surplus.

Otherwise, explicitly accept the existing policy: treasury revenue may be paid while lenders still have an unrecovered principal loss.

Lattica: Resolved with [@Oaefdb025a...](#)

Zenith: Verified.

[M-5] Renewal can keep debt above the current collateral limit

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [BorrowerLogic.sol#L29-L51](#)
- [BorrowerLogic.sol#L96-L120](#)
- [BorrowerLogic.sol#L147-L153](#)
- [lattica-protocol-spec.md#L56-L62](#)

Description:

A new loan uses a fresh collateral price to calculate how much can be borrowed. Renewing an existing loan is also described as fresh underwriting, so a lower collateral value should reduce the amount eligible for renewal.

Renewal instead checks the new principal against the old debt, without requiring a fresh collateral price. With a valid quote for that old amount, a borrower can start another term even though a new loan against the same collateral would exceed the current limit.

The code below compares the new principal only with the stored debt (`l.principal`). It then renews the loan without recalculating how much the collateral supports at its current price:

```
if (q.principal != l.principal) {  
    revert Errors.PrincipalOutOfBand(l.principal, q.principal);  
}  
premiumOracle.consumeQuote(q, qSig);  
newLoanId = ledger.recordRoll(loanId, q);
```

For example:

1. The borrower pledges 10,000 tokens priced at 0.60 pUSD. At a 50% borrowing limit, the original loan is 3,000 pUSD.
2. The price falls to 0.40 pUSD. Those tokens now support only 2,000 pUSD of new borrowing.
3. The borrower obtains a valid renewal quote for the existing 3,000 pUSD debt and pays the new fees.

4. Renewal accepts the old debt amount, so another term starts with 1,000 pUSD above the current collateral-based limit.

Recommendations:

Require a fresh collateral mark for renewal and apply the current collateral-based borrowing limit. If existing debt exceeds that limit, require the borrower to repay the excess before renewing.

Lattica: Resolved with [@df16a1ea9c...](#)

Zenith: Verified.

[M-6] Renewal is rejected when the existing loan already fills the budget

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [BorrowerLogic.sol#L102-L120](#)
- [Configurator.sol#L128-L147](#)
- [Ledger.sol#L204-L230](#)

Description:

An exposure budget limits how much debt the pool may carry. Renewing a loan for the same principal replaces its term without increasing that debt, so it should not require room for a second copy of the loan.

The renewal entry point checks for room for the full new principal while the old principal still occupies the budget. It rejects the request before reaching the later accounting that releases the old amount and replaces it with the same amount.

The code below asks for headroom for the full successor principal before recording the renewal. The old loan still occupies that budget at this point:

```
configurator.requireOriginationAllowed(l.tokenId, q.principal);  
newLoanId = ledger.recordRoll(loanId, q);
```

For example:

1. A market has a 3,000 pUSD budget, fully occupied by one 3,000 pUSD loan.
2. The borrower requests renewal of the same amount, with a valid quote and enough money for the new fees.
3. The check sees no unused budget and asks for another 3,000 pUSD, so it rejects renewal.
4. If replacement accounting were reached, debt would remain 3,000 pUSD and stay within the budget.

The same rejection can occur at the market, cluster or pool budget. The borrower cannot renew through this path even when all other conditions are met. The failed request leaves the original loan unchanged.

Recommendations:

Since this entry point already requires the new principal to equal existing debt, check zero additional exposure at the early gate. Keep the market, breaker and resolution checks, and keep the Ledger's final release-and-consume budget enforcement:

```
configurator.requireOriginationAllowed(l.tokenId, q.principal);  
configurator.requireOriginationAllowed(l.tokenId, 0);
```

Lattica: Resolved with [@7663a84f2e...](#)

Zenith: Verified.

[M-7] Temporary liquidity can lock in cheaper interest for a loan epoch

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/BorrowerLogic.sol](#)
- [src/InterestRateManager.sol](#)
- [src/LatUSD.sol](#)
- [src/Ledger.sol](#)

Description:

Loan pricing treats newly deposited liquidity as immediately available when choosing a fixed interest rate, without tying that benefit to the deposit's continued presence. In Ledger, utilization divides outstanding principal by tracked assets. A real pUSD deposit through LatUSD increases idle assets and therefore the denominator. Where the configured interest curve rises with utilization, this reduces the rate returned by InterestRateManager.

BorrowerLogic reads that spot rate when calculating origination charges. Ledger then records the same rate on the new loan and fixes the lender income stream for its epoch. The vault's redemption machinery has no corresponding adjustment to these terms: fulfillment removes idle assets from the accounting, and the subsequent claim transfers the assigned pUSD to the controller. A borrower who also supplies liquidity can consequently recover that capital while retaining the cheaper loan terms. The effect applies to freshly priced debt for the affected epoch; it does not alter rates on loans already outstanding.

This requires genuine funding and an otherwise admissible loan, including eligible collateral, valid signed underwriting and marks, and sufficient budget headroom. Existing principal must be nonzero, and a substantial rate reduction requires a deposit large relative to the vault's asset base. Recovering the temporary deposit promptly also depends on the original idle balance covering the loan draw, sufficient redemption liquidity, and an empty or clearable FIFO queue. Requests must satisfy the minimum share amount. The calls can complete sequentially, but capital that remains queued cannot be assumed available for an atomic exit or repayment of short-term financing.

The economic benefit is the reduction in epoch interest, offset by funding costs, transaction fees, rounding, and the cost of any extended wait for redemption. Lenders and the Buffer receive less interest revenue according to their respective shares; the borrower still owes principal and pays the required premium. The utilization cap remains an admission check,

and later redemptions are expressly allowed to raise utilization above it. Actual savings depend on the deployed curve and available capital, while exit timing depends on live liquidity and queue conditions.

Recommendations:

Enforce pricing and redemption rules that prevent brief capital additions from securing a lasting interest discount across every path that fixes new loan terms.

Lattica: Resolved with [@9754cd6d0b....](#)

Zenith: Verified.

[M-8] Leverage reservations count the principal twice when checking available liquidity

SEVERITY: Medium

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/LatUSD.sol](#)
- [src/Ledger.sol](#)
- [src/Leverage.sol](#)

Description:

A leverage request can have enough cash available for its principal and still fail during payment. The reservation and vault apply their liquidity checks to different accounting states, causing the requested principal to be counted twice.

In `reserveOpen`, the executor first submits the intent to `Ledger.openReservation`. The ledger verifies the draw against `idle`, creates the reservation, and moves its principal from `idle` to `totalPrincipalOutstanding`. Only afterward does the Pool collect the trader's margin and ask `LatUSD.disburse` to pay the principal. That payment checks `availableForOrigination`, which subtracts redemption queue liability from the ledger's current `idle` balance. The principal has already been removed from that balance. The intervening margin transfer goes to the Pool and does not replenish vault `idle`.

Let P be the requested principal, I the idle balance immediately before the reservation debit, and Q the queue liability. Funding the draw while retaining the queue's cash requires $P + Q \leq I$. The implemented payment check instead requires $P \leq I - P - Q$, or $2P + Q \leq I$. Consequently, a request satisfying the first bound but exceeding the second reverts with `ExceedsAvailableForOrigination`. This remains possible after a valid signed quote, unused COID, sufficient approved margin, market eligibility, budget headroom, and utilization checks have all passed. It occurs during routine execution by the authorized executor and does not require a compromised role. Fulfillment of a lender's own redemption request before execution can also reduce the idle balance enough to move a pending request into this interval; the lender receives only the cash attributable to those shares.

The failed transaction costs gas and postpones execution at the requested size. Its revert also undoes quote consumption, reservation accounting, and the margin transfer, leaving no trapped margin or reservation and causing no principal loss. The restriction applies to each attempted draw: separately quoted smaller requests or additional accounted liquidity can permit execution, so it does not impose a permanent 50% limit on aggregate utilization

or block exits. The rejection follows deterministically from these balances; how frequently live request sizes encounter it is unknown.

Recommendations:

Ensure every leverage reservation and its disbursement use a consistent liquidity basis that counts the principal once while preserving cash owed to the redemption queue.

Lattica: Resolved with [@2dbff51c58...](#)

Zenith: Verified.

[M-9] Repayment after cluster reassignment removes unrelated exposure from budget accounting

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/BorrowerLogic.sol](#)
- [src/Configurator.sol](#)
- [src/Ledger.sol](#)

Description:

Cluster limits depend on the exposure recorded in `Configurator._consumed`. Both borrowing and repayment select the cluster through the market's current `clusterOf` value. However, `setCluster` changes that value without transferring the consumption associated with loans already open in the market. The interface describes cluster re-estimation as an epoch operation, and the setter permits the risk manager or timelock to perform it while exposure is live.

This creates a mismatch when an authorized, routine reassignment is followed by repayment. The public `repay` entry point accepts payment from any payer and invokes `Ledger.recordRepay`, which releases the repaid amount using the loan's token ID. `releaseBudget` then resolves the market's new cluster rather than the cluster charged when the loan originated. Its saturating subtraction removes up to the repaid amount from the new cluster's aggregate counter. If that counter contains consumption belonging to other markets, those entries can be reduced or entirely erased even though their loans remain outstanding. The original cluster retains its debit.

The destination can therefore appear to have room for another loan that should fail its concentration limit. Both `requireOriginationAllowed` and `consumeBudget` consult the understated counter, so a subsequent ordinary borrow can pass both checks. Borrowers gain access to credit beyond the intended cluster allocation, lenders lose an accurate enforcement of correlated exposure limits, and markets remaining in the original cluster may have less borrowing capacity than their actual exposure warrants. A financial loss to lenders still depends on adverse performance of the resulting correlated exposures.

The borrower cannot initiate the privileged reassignment and must supply the actual repayment funds. Using the apparent capacity also requires a normally issued quote, valid signatures, sufficient collateral, and available cash; utilization, market budgets, and the pool budget continue to constrain the loan. The relevant state requires an outstanding loan in

the reassigned market and separately recorded consumption in its destination. For each erroneous release, the destination counter falls by no more than the principal returned or the consumption present there. This is a bounded distortion of cluster accounting, not debt forgiveness or a collateral transfer to the payer.

Changing a budget ceiling does not reconcile the mismatch: `setBudget` writes `_budget` without correcting `_consumed`. Although the release comments acknowledge the stale source debit and refer to periodic budget reseeding, this does not provide an accounting repair or justify removing consumption for unrelated loans in the destination.

Recommendations:

Enforce consistent attribution of outstanding exposure across every cluster reassignment, repayment, and subsequent origination check. Ensure repayments remain available while releasing only the consumption attributable to the repaid exposure, preserving the recorded exposure of unrelated loans.

Lattica: Fixed in [commit 2af5aeb](#).

Zenith: Verified.

[M-10] Capped recoveries make the loss ratio depend on settlement order

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [src/Configurator.sol](#)
- [src/Ledger.sol](#)
- [src/Liquidator.sol](#)

Description:

Recovery accounting can produce different final loss ratios for the same completed settlements and cash movements. In `Liquidator.remitProceeds`, proceeds first reimburse the Buffer advance, and the remaining amount is split between treasury and pool. The entire pool portion enters the vault and is credited through `recordDeposit`. However, only the smaller of that portion and the current `realizedLosses` is passed to `recordRecovery`. Any excess contributes to assets without being retained as a credit against subsequent losses.

A separate loan can add to the same loss counter through the public `claimExpired` entry point. The loan must still be active and have reached its expiry; settlement then books the principal left uncovered by Buffer cash and available escrow. If recovery is processed while the loss counter is too small to absorb it, some of its statistical credit is discarded. Processing the eligible loss first instead allows more of that same recovery to reduce the counter. Both operations can therefore finish with identical pool cash and NAV but different `realizedLosses`. The recovery adjustment itself has no NAV effect, and the same settled loans contribute the same premiums to the ratio's denominator.

An ordinary caller can influence this outcome by ordering an eligible expiry claim relative to an honest remittance for an already settled loan. The remitter role and actual payment are still required. The opportunity depends on recovery exceeding the losses available to offset it, another expired loan with an uncovered balance, and successful transaction ordering. Existing exposure can supply the eligible loan, so the caller may need only gas rather than fresh collateral or recovery capital; the necessary events must nevertheless coincide.

`Configurator.requireOriginationAllowed` compares `realizedToPredicted` against `breakerBound`. This check governs borrowing, loan renewals, and leverage reservations. Where the resulting ratios straddle the bound and the prior breaker state permits a difference, ordering can obstruct admission or suppress the loss signal used to stop new credit. Pool users consequently receive inconsistent credit decisions despite the same

final assets. Repayment and withdrawal paths do not use this admission gate, and the accounting discrepancy alone does not remove funds.

`pokeBreaker` can latch an above-bound ratio, after which the timelock must clear the flag. Clearing it does not restore admission while the live ratio still exceeds the bound, so the restriction's duration depends on subsequent recovery or governance action. The persistent difference in the underlying metric exists independently of this intended latch behavior; a manual breaker can stop admissions but cannot correct the ordering-dependent statistic.

Recommendations:

Ensure cumulative loss accounting preserves eligible recovery credit across every recovery and loss settlement, so equivalent completed events yield the same ratio regardless of transaction order. Enforce that invariant throughout the metric consumed by all admission checks and breaker decisions.

Lattica: Fixed in [commit 63e02f4](#).

Zenith: Verified.

4.3 Low Risk

A total of 5 low risk findings were identified.

[L-1] Recovery after all shares exit can trap the next depositor below the redemption floor

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Liquidator.sol#L140-L176](#)
- [LatUSD.sol#L118-L130](#)
- [LatUSD.sol#L152-L164](#)
- [LatUSD.sol#L196-L226](#)
- [MathLib.sol#L124-L135](#)
- [vault-design.md#L120-L120](#)

Description:

The vault uses `minRequest` as a fixed minimum share amount for redemption requests. This is meant to keep dust requests out of the FIFO exit queue. The minimum is measured in shares, not in pUSD value.

A normal recovery can arrive after all old LP shares have been redeemed and burned. When the remitter returns proceeds, the pool share is transferred into the vault and recorded as vault assets:

```
pusd.safeTransferFrom(msg.sender, address(vault), poolShare);
ledger.recordDeposit(poolShare);
uint256 losses = ledger.realizedLosses();
uint256 netted = poolShare < losses ? poolShare : losses;
if (netted != 0) ledger.recordRecovery(loanId, netted);
```

A later deposit mints shares against the current tracked assets and total supply. If real supply is zero but tracked assets are already present, the share-conversion formula can mint very few shares for a normal pUSD deposit:

```
return Math.mulDiv(assets, totalShares + 10 ** offset, totalAssets + 1,  
    Math.Rounding.Floor);
```

The redemption request then checks only the raw share amount before it creates a queue entry:

```
if (shares < minRequest) revert Errors.BelowMinRequest(shares, minRequest);
```

For example:

1. All old LP shares have exited, so real `totalSupply` is zero.
2. A later recovery sends 200 pUSD of pool share into the vault.
3. A new LP deposits 1,000 pUSD.
4. The share formula uses $1,000e6 * 1e12 / (200e6 + 1)$, so the new LP receives about $5e12$ raw shares, which is 0.000005 share with 18-decimal shares.
5. If `minRequest` is 10 shares, the user's whole position is below the request floor.
6. The vault has cash and the user has a normal economic balance, but `requestRedeem()` rejects before any queue entry exists.

This needs the old supply to be fully burned before the recovery lands, and the fixed `minRequest` must remain above the new share base.

Recommendations:

Do not record pool recovery into vault assets when `totalSupply() = 0`. Revert that pool-share leg or route it to a separate reserve until there is share supply to receive it.

Lattica: Resolved with [@2a43da5fac...](#)

Zenith: Verified.

[L-2] A zero-premium loss can leave the automatic breaker open

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Ledger.sol#L674-L676](#)
- [Configurator.sol#L129-L133](#)
- [Ledger.sol#L692-L698](#)
- [Ledger.sol#L742-L745](#)

Description:

The automatic breaker is meant to stop new originations when realized losses exceed the risk that was priced through premiums. The origination gate reads `realizedLosses / settledPremiums` before allowing a new borrow or leverage reservation.

The zero-denominator branch returns 0 even when a loss has already been recorded. If a settled losing loan has zero premium, `settledPremiums` remains zero and `realizedToPredicted()` reports a healthy value instead of a value above the breaker threshold:

```
function realizedToPredicted() external view returns (uint256) {
    if (settledPremiums == 0) return 0;
    return MathLib.wadDivDown(realizedLosses, settledPremiums);
}
```

The origination gate uses this returned value directly:

```
if (ledger.realizedToPredicted() > breakerBound) revert
    Errors.BreakerActive();
```

A zero premium can be recorded because the premium is computed from the signed `premiumBps` value and there is no positive lower bound:

```
uint256 premium = MathLib.bpsUp(q.principal, q.premiumBps);
```

When the loan later settles, only the premium amount is added to the breaker's denominator:

```
settledPremiums += l.premium;
```

Example sequence:

1. The quote signer signs a 3,000 pUSD direct-loan quote with `premiumBps = 0`.
2. The borrower opens the loan. Interest can still be charged, but the premium stored for the breaker is 0.
3. The loan later expires unpaid and records a realized loss.
4. The protocol now has positive `realizedLosses` and zero `settledPremiums`.
5. The breaker ratio still returns 0, so the automatic gate treats the book as healthy and can allow a later origination.

This requires a valid zero-premium quote. A borrower cannot create that signature by themselves. The issue is the onchain zero-denominator rule: once losses are positive, a zero prediction denominator should not report a healthy value.

Recommendations:

Return a value above the breaker threshold when `realizedLosses > 0` and `settledPremiums = 0`. For example, keep returning 0 when both values are zero, but return `type(uint256).max` once a loss exists with a zero denominator. Alternatively, reject zero-premium quotes for positive-principal originations.

Lattica: Resolved with [@0c1333248b...](#)

Zenith: Verified.

[L-3] A delayed expiry claim lets the first LP redeem at an inflated price

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Ledger.sol#L301-L309](#)
- [Ledger.sol#L548-L555](#)
- [Ledger.sol#L656-L685](#)
- [LatUSD.sol#L196-L209](#)
- [LatUSD.sol#L238-L269](#)
- [Ledger.sol#L786-L794](#)

Description:

Each loan or leverage position creates an income stream with a fixed end time. The stream should stop contributing to the share price at that time. However, `Ledger.accrue()` adds the global stream rate until the current timestamp, even when a stream has already expired:

```
if (ts > _lastAccrual) {
  _indexRay += MathLib.mulDivDown(_ratePerSecondRay, ts - _lastAccrual,
    1);
  _lastAccrual = ts;
}
```

`totalAssets()` uses this index when it prices shares. `requestRedeem()` calls `accrue()` before the request is fulfilled, so a lender can exit while the expired stream is still adding value. The later expiry path corrects the index only when someone calls it:

```
if (ts > s.end) {
  _indexRay -= MathLib.mulDivDown(s.rateRay, ts - s.end, 1);
  ts = s.end;
}
```

The correction happens after the first lender has already received cash. The remaining lenders absorb the correction.

Example sequence:

1. Alice and Bob each deposit 50,000 pUSD, so the pool has 100,000 pUSD of NAV and each owns half of the shares. The pool also has enough idle cash to pay one lender's exit.
2. Carol borrows 3,000 pUSD for 24 hours. The loan's lender income is 100 pUSD, so the stream should add 100 pUSD in total and then stop.
3. The 24-hour term ends, but no one calls the expiry claim. Thirty days after origination, Alice submits `requestRedeem()` for all her shares. That call invokes `accrue()` first. The stream rate is 100 / 24 pUSD per hour, so the index now counts 3,000 pUSD instead of the correct 100 pUSD.
4. The pool therefore prices Alice's half at about 51,500 pUSD instead of the fair 50,050 pUSD. Alice completes her redemption and receives the inflated amount.
5. A keeper then calls the expiry claim. The contract removes the extra 2,900 pUSD from the index and leaves only the earned 100 pUSD. Bob's equal share is now worth about 48,600 pUSD, so Bob loses about 1,450 pUSD compared with the fair result.

The result is a transfer from the LP who exits before the expiry claim to the LPs who remain afterward. The same accounting applies to both direct-loan streams and leverage-position streams.

Recommendations:

Cap every stream's contribution at `min(block.timestamp, s.end)` before the value is used for share pricing. If the global index is kept, update or settle expired streams before `requestRedeem()` and `_crank()` can use the index.

If the protocol intentionally relies on `claimExpired()` being called promptly, state that operational dependency explicitly and record this issue as an accepted design risk. In that case, LP exit value depends on how quickly the expiry claim is called.

Lattica: Resolved with [@b01961488e...](#)

Zenith: Verified.

[L-4] Leverage advances cannot be reconciled through the reserve repayment path

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [src/Buffer.sol](#)
- [src/Ledger.sol](#)
- [src/Leverage.sol](#)
- [src/Liquidator.sol](#)

Description:

Buffer records reserve contributions for both direct loans and leveraged positions, but its repayment mechanism only supports the direct-loan lifecycle. An ordinary loss-making leverage close can therefore create an outstanding advance that no implemented recovery entry point can reduce.

When the executor closes a filled, open position, the settlement waterfall applies trader margin and the position's remaining escrowed income before drawing reserve cash. If a deficit remains and Buffer has funds, the Pool calls cover with the position ID. Buffer transfers the covered amount to the vault and increases `outstandingAdvance` for that ID. A successful close leaves the position marked `Closed` in Ledger.

The only function that decreases the advance is `Buffer.repayAdvance`, and it accepts calls exclusively from `Liquidator`. `Liquidator` reaches that function through `remitProceeds`, which first checks that `ledger.loan` reports either `Liquidated` or `Expired`. Ledger allocates loans and leverage reservations from one counter while storing their records in separate mappings; a completed reservation becomes a position under its existing ID. Consequently, the direct-loan record for that position ID remains `None`, even after the position closes. The remittance status check rejects it before repayment can occur, and calling Buffer directly cannot bypass the caller restriction.

This leaves the position's advance unreconciled if later recovery needs to be accounted for, potentially accumulating discrepancies in outstanding-advance monitoring. It arises during honest executor settlement; public callers do not control the close or remittance parameters. Backend recovery practices and the intended handling of these position advances are not established by the contracts.

Reserve cash can still be replenished through prime or token transfers, although neither

changes the recorded advance. Available coverage and reserve health depend on the actual token balance. The original payout is the intended absorption of the position's loss: this mismatch does not add trader debt, inflate lender NAV, or trap reserve funds.

Recommendations:

Ensure every advance created for a leverage position has an authorized reconciliation path that recognizes the position's lifecycle and updates its outstanding amount consistently with repayment. Preserve the distinction between loan and position records across all reserve recovery entry points.

Lattica: Fixed in [commit 8366980](#).

Zenith: Verified.

[L-5] A negligible early default can latch the pool's admission breaker

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [src/Buffer.sol](#)
- [src/Configurator.sol](#)
- [src/Ledger.sol](#)
- [src/Liquidator.sol](#)

Description:

A small unpaid loan can interrupt new business throughout a pool when its settled-premium history is still very small. Ledger's `realizedToPredicted` returns the ratio of `realizedLosses` to `settledPremiums`, or zero when no premium has settled. Configurator compares that ratio with `breakerBound` without also considering the absolute loss or whether the premium sample is meaningful. A negligible uncovered amount can therefore produce a threshold crossing disproportionate to its monetary size.

On expiry, the permissionless `claimExpired` path settles the loan using Buffer cash and available escrowed income as coverage. Ledger adds the loan's premium to its settled total and records the principal left uncovered as realized loss. If an honestly underwritten, sufficiently small loan goes unpaid while little other premium has settled, its premium and residual shortfall can produce a large ratio despite the small amounts involved. The borrower forfeits the posted collateral and incurs transaction costs; this disruption does not require a dishonest price or quote signer.

The ratio crossing immediately causes `requireOriginationAllowed` to reject new borrowing, loan rolls, and leverage reservations. Anyone can then invoke `pokeBreaker` to set the persistent breaker flag. Later collateral proceeds may reduce the loss statistic and improve the ratio, but the flag remains until the timelock clears it. Clearing the flag while the ratio still exceeds the bound leaves admission refused by the live check. The resulting interruption affects that pool's prospective business; the breaker does not gate repayment, redemption, or expiry claims, and does not shut down independent deployments.

This condition depends on the honest quoting service accepting a sufficiently small loan, a valid collateral mark, a listed market, available origination capacity, inadequate settlement coverage, and a suitable breaker bound and timing. The borrower cannot erase previously accumulated premiums or withdraw reserve funds to arrange these conditions. Operational

reachability therefore depends on signer size policy, reserve capitalization, and the pool's early configuration. The concern is the outsized admission effect of a negligible early loss, rather than the ordinary protective response to a substantial default.

The booked shortfall may subsequently be recovered through collateral disposal, so the admission interruption does not establish a final lender loss or a profitable strategy for the borrower. Its duration depends on recovery and timelock intervention; no fixed response time or permanent shutdown follows from this path.

Recommendations:

Ensure negligible defaults in an immature premium sample cannot independently force a pool-wide admission stop, across both live admission checks and persistent ratio-based breaker activation. Preserve effective protection against material deterioration and a controlled process for restoring admission after the triggering condition is resolved.

Lattica: Resolved with [@80d522a47d....](#)

Zenith: Verified.